왜 레디스인가 — 발표 대본

Nov 13, 2026 · @Velocity

이 대본 쓰는 법

쪽 번호는 수정된 PDF(24쪽) 기준입니다. 전체 20~25분 분량이고, 굵은 글씨는 꼭 짚을 문장입니다.

시뮬레이터(로컬 캐시가 망가지는 다섯 장면)는 6~10쪽 문제 장면에서 씁니다. 각 장면마다 슬라이드로 상황을 말한 뒤, 시뮬레이터로 넘어가 직접 눌러 보여 주고, 그 장면에서 바로 "레디스로 바꾸면"까지 보여 줄지, 아니면 14쪽 해결 슬라이드에서 몰아서 보여 줄지 정하면 됩니다. 이 대본은 14쪽에서 몰아서 보여 주는 흐름으로 썼습니다. 주소 끝을 #scene-1 ~ #scene-5로 바꾸면 해당 장면이 바로 열립니다.

1~2쪽 · 오프닝

1쪽 · 왜 레디스인가

오늘 주제는 "레디스가 무엇인가"보다 "레디스가 왜 필요해졌나"입니다. 결론부터 말씀드리면 한 줄입니다. **로컬 캐시는 상태를 각 서버 메모리에 묶어 두고, 레디스는 그 상태를 서버 밖의 빠른 공용 저장소로 빼냅니다.** 지금은 이 말이 잘 안 와닿으실 텐데, 정글마켓이라는 쇼핑몰 하나를 끝까지 따라가면서 이 문장이 왜 맞는지 보겠습니다.

2쪽 · 레디스는 "고통"에서 출발했다

오늘 이야기는 다섯 단계입니다. 로컬 캐시는 좋은 도구인데, 서버가 여러 대가 되는 순간 다섯 가지 문제가 터집니다. 그 다섯 개를 따라가 보면 원인은 하나로 모이고, 그 원인을 고치는 방법이 레디스입니다.

여기서 두 가지만 기억해 주세요. **서버가 여러 대라는 건 "조건"이지 "원인"이 아닙니다.** 같은 코드도 서버 한 대면 멀쩡합니다. 그리고 다섯 증상을 하나씩 막으면 끝이 안 나고, 뿌리를 고쳐야 다섯 개가 한 번에 풀립니다.

3~5쪽 · 출발점과 조건

3쪽 · 로컬 캐시는 빠르고 좋은 도구다

정글마켓에서 상품 하나를 DB에서 읽으면 200ms 걸립니다. 그래서 한 번 읽은 상품은 서버 메모리의 딕셔너리, 즉 `cache` 라는 변수에 넣어 둡니다. 다음부터는 0.01ms면 꺼냅니다. 네트워크를 안 타니 가장 빠르고, DB에 가는 요청이 줄어서 DB도 보호됩니다.

코드를 보시면, 캐시에 있으면 바로 돌려주고 없을 때만 DB에 갑니다. **이 "있으면 DB를 안 본다"는 줄을 기억해 두세요.** 뒤에서 문제의 씨앗이 됩니다. 그래도 서버가 한 대라면 여기서 끝입니다. 레디스는 필요 없습니다.

4쪽 · 서버가 여러 대가 된다

그런데 쇼핑몰이 커지면 서버를 여러 대 둘 수밖에 없습니다. 세일 날 접속자가 10배로 늘면 한 대를 키우는 데 한계가 있고, 한 대가 죽으면 쇼핑몰 전체가 멈추고, 배포할 때도 한 대씩 바꿔야 멈추지 않습니다.

그림을 보시면 서버 A, B, C가 **똑같은 코드를 따로 실행하고 있고, 각자 자기 메모리에 cache와 sessions를 들고 있습니다.** 이름은 같은 변수지만 세 개가 따로 있는 겁니다. 그리고 그 앞에 로드밸런서가 있습니다.

5쪽 · 로드밸런서는 요청을 나눠 주는 안내원이다

로드밸런서는 마트 계산대 앞에서 손님을 비어 있는 계산대로 안내하는 직원 같은 겁니다. 사용자는 주소 하나로 접속하고, 로드밸런서가 서버를 골라 줍니다. 기본 방식은 라운드 로빈, 즉 A, B, C 순서대로 돌아가며 나눠 주는 겁니다. 실무에서 AWS나 Nginx도 따로 설정하지 않으면 이 방식입니다.

여기서 중요한 건 두 가지입니다. 첫째, **요청 하나는 서버 한 대에만 갑니다.** 로드밸런서는 요청을 복사해서 세 대에 다 보내지 않습니다. 둘째, **로드밸런서는 누가 누구인지 기억하지 않습니다.** 그래서 kim의 첫 클릭은 A로, 다음 클릭은 B로 갈 수 있습니다. 실제로는 다른 사용자 요청이 사이사이 섞이니까, kim 입장에선 거의 무작위로 서버가 바뀝니다.

그럼 A와 B가 기억하는 게 서로 다르면 어떻게 될까요? 지금부터 다섯 장면을 보겠습니다.

6~10쪽 · 문제 장면 다섯 개

6쪽 · 장면 ① 새로고침할 때마다 가격이 바뀐다

키보드가 50,000원이고, 서버 세 대 모두 이미 한 번씩 조회해서 캐시에 50,000원을 들고 있습니다. 관리자가 40,000원으로 할인합니다. 이 요청은 서버 한 대, 마침 차례였던 A로 갑니다.

A는 DB를 40,000원으로 고치고 자기 캐시도 고칩니다. 그런데 **캐시는 A 프로그램 안의 변수일 뿐이라, A는 B와 C의 메모리에 손댈 방법이 없습니다.** 여러분 노트북에서 변수를 바꿔도 옆 사람 노트북의 같은 이름 변수는 안 바뀌는 것과 같습니다.

그럼 B와 C도 DB를 보면 되지 않느냐? 아까 기억해 두라고 한 줄이 여기서 나옵니다. **캐시에 값이 있으면 DB를 안 봅니다.** B와 C는 50,000원을 이미 들고 있으니 DB가 바뀐 줄 모르고 계속 그걸 줍니다. 그래서 고객이 새로고침할 때마다 40,000원, 50,000원이 번갈아 보입니다.

> 시뮬레이터 #scene-1: 새로고침 두세 번 → 할인 → 새로고침 여러 번. 빨갛게 변하는 서버 칸과 고객 화면의 가격 기록을 보여 줍니다.

7쪽 · 장면 ② 로그인했는데 다음 클릭에서 로그아웃된다

kim이 로그인하면 서버가 abc라는 세션 번호를 만들어 브라우저에 쿠키로 주고, 자기 메모리에 "abc는 kim"이라고 적습니다. 이 요청이 A로 갔다면 그 메모는 A에만 있습니다.

kim이 마이페이지를 누르면 브라우저는 쿠키 abc를 그대로 들고 갑니다. 그런데 로드밸런서가 B로 보내면, B 메모리에는 abc가 없습니다. **쿠키는 번호표일 뿐이고, 그 번호표가 누구 것인지는 서버 메모리에 적혀 있습니다.** B 입장에선 처음 보는 번호표라 "로그인하세요"가 뜹니다.

> 시뮬레이터 #scene-2: 로그인 → 마이페이지 서너 번.

8쪽 · 장면 ③ 배포할 때마다 DB가 휘청인다

평소엔 캐시가 다 차 있어서 DB 요청이 초당 100 정도입니다. 새 버전을 배포하려고 A, B, C를 재시작하면, 캐시는 프로그램 안의 변수였으니 같이 사라집니다. 캐시 세 개가 동시에 텅 비고, 모든 요청이 DB로 몰려서 초당 5,000이 됩니다.

세일 날 서버 D를 추가해도 D는 빈 캐시로 시작하니 오히려 DB 부하가 늘어납니다. **캐시의 목적은 DB 보호인데, 가장 위험한 순간에 DB를 못 지킵니다.**

> 시뮬레이터 #scene-3: 트래픽 흘리기 → 배포하기 → 서버 D 추가. 그래프가 치솟는 순간을 보여 줍니다.

9쪽 · 장면 ④ 같은 데이터를 세 번 저장하고, DB에 세 번 묻는다

인기 상품 1,000개를 캐시에 올리려면, A도 1,000번 DB에 묻고, B도 똑같이 1,000번, C도 1,000번 묻습니다. 내용은 셋 다 똑같습니다. 서버가 10대면 메모리도 10배, DB 조회도 10배입니다. 그리고 캐시 크기는 각 서버 메모리를 넘을 수 없습니다.

> 시뮬레이터 #scene-4: 요청 8개 보내기 세 번 → 상품 8개인데 DB 질문 24번, 사본 24개.

10쪽 · 장면 ⑤ 함께 세야 하는 값은 아예 만들 수 없다

한정판 키보드가 1개 남았습니다. kim은 A로, lee는 B로 거의 동시에 구매합니다. A는 자기 메모리를 보니 재고 1개, 팝니다. B도 자기 메모리를 보니 재고 1개, 팝니다. **1개를 2명에게 팔았습니다.** 서로의 판매를 모르기 때문입니다.

오른쪽 표처럼 조회수, 선착순 쿠폰, 중복 결제 방지, 로그인 5회 실패 잠금도 전부 같은 문제입니다. 선착순 100명 쿠폰은 서버마다 100명씩 세서 300명이 받습니다. 앞의 네 장면은 "틀리거나 느려지는" 문제였다면, **이건 기능 자체를 만들 수 없는 문제**입니다.

> 시뮬레이터 #scene-5: 동시 구매 → 쿠폰 10명 클릭. 3장 한도인데 9장이 나가는 걸 보여 줍니다.

11~13쪽 · 뿌리

11쪽 · 다섯 장면의 원인은 하나다

다섯 장면이 다 달라 보이지만 원인은 하나입니다. **상태가 각 서버 프로세스의 메모리에 묶여 있다.** 가격은 서버마다 캐시가 달라서, 로그아웃은 서버마다 세션이 달라서, 배포는 재시작하며 상태가 같이 사라져서, 중복은 같은 상태를 따로 들고 있어서, 재고는 모두가 봐야 할 상태를 둘 곳이 없어서 생긴 문제입니다.

여기서 상태란, 같은 요청인데도 답을 다르게 만드는, 서버가 기억하고 있는 데이터입니다.

12쪽 · 상태가 서버를 stateful로 만든다

코드를 보시면 똑같이 `mypage("abc")`를 불러도, sessions가 비어 있으면 "로그인하세요", abc가 있으면 "kim님 마이페이지"가 나옵니다. 요청은 똑같은데 sessions에 뭐가 있느냐에 따라 답이 바뀌죠. 이렇게 자기만 아는 상태를 들고 있어서 요청이 "그 서버"로 가야만 제대로 처리되는 서버를 stateful이라고 합니다. 반대로 상태를 안 들고 있어서 어느 서버로 가도 같은 답을 주면 stateless입니다.

아래 표의 차이도 짚어 주세요. 세션은 그 서버에만 있는 원본이라 서버가 꺼지면 다시 로그인해야 합니다. 기능이 깨지는 거죠. 캐시는 원본이 DB에 있는 사본이라 서버가 꺼져도 잠깐 느려질 뿐입니다.

13쪽 · 하나를 막으면 다른 게 남는다

그럼 증상을 하나씩 막으면 안 될까요? 실제로 다 시도된 방법들입니다.

캐시 TTL을 5초로 짧게 하면 불일치 시간이 짧아지지만, 5초 동안은 여전히 다르고, 짧을수록 DB 요청이 늘어서 장면 ③이 심해집니다. 오래된 키를 지우면 메모리는 아끼지만 구조는 그대로입니다. 같은 사용자를 같은 서버로 보내는 스티키 세션은 로그아웃을 막지만, 그 서버가 죽으면 세션도 사라지고 부하가 한쪽에 몰립니다. 로그인 정보를 모든 서버에 복사하는 세션 복제도 있었는데, 서버가 늘수록 복사 트래픽이 폭증하고, 서버마다 전체 세션을 들고 있어서 장면 ④를 세션에서 다시 겪고, 복사가 끝나기 전엔 여전히 로그인이 풀립니다.

표의 오른쪽 열을 보시면 전부 "그대로"가 남습니다. **증상이 아니라 뿌리를 고쳐야 합니다. 상태를 서버에서 떼어내는 겁니다.**

14~15쪽 · 해결

14쪽 · 상태를 서버 밖 한 곳으로 뺀다

그림을 4쪽과 비교해 보세요. 서버 A, B, C에는 이제 코드만 있고 상태가 없습니다. cache, sessions, 재고, 조회수는 전부 서버 밖의 공용 저장소 한 곳에 있습니다.

그러면 다섯 장면이 한 번에 풀립니다. 가격을 바꾸면 공용 저장소의 키 하나만 지우면 되니 모두 새 가격을 봅니다. 세션이 공용 저장소에 있으니 어느 서버로 가도 kim입니다. 저장소는 앱과 따로 살아 있으니 앱을 재시작해도 캐시가 남습니다. 한 벌만 저장하고 DB에도 한 번만 묻습니다. 모두가 같은 재고 키를 보니 1개면 한 명만 삽니다.

> 시뮬레이터: 오른쪽 위 "레디스로 바꾸면"을 누르고, 장면 ①②③⑤를 아까와 똑같이 눌러 봅니다. 서버 칸이 "상태 없음 · 코드만"으로 바뀌고 빨간 레디스 칸이 생기는 것, 같은 조작인데 결과가 멀쩡한 것을 보여 줍니다. 시간이 부족하면 ①과 ⑤만 보여 줘도 충분합니다.

15쪽 · 그 공용 저장소를 어디에 둘까

후보는 세 곳입니다. 앱 서버 안에 두면 그게 로컬 캐시라 처음 문제로 돌아갑니다. DB에 두면 공유는 되지만 느리고 늘리기 어렵고, 무엇보다 DB를 보호하려던 캐시를 DB에 두는 건 목적과 정반대입니다. 그래서 **메모리 기반의 별도 서버**가 답입니다. 공유되고, RAM이라 빠르고, DB 앞에서 요청을 막아 줍니다. 그게 레디스입니다.

리액트를 아시는 분이라면, 리액트가 상태와 화면을 분리했듯 레디스는 상태와 서버를 분리한다고 생각하시면 됩니다.

16~18쪽 · 레디스 이전과 레디스

16쪽 · Memcached는 공유를 얻고 자료구조를 잃었다

사실 "메모리 기반 별도 서버"는 레디스가 처음이 아닙니다. 2003년에 Memcached가 있었고, 공유는 성공했습니다. 문제는 값으로 문자열 하나만 넣을 수 있다는 점입니다.

판매 랭킹에 키보드 점수를 1 올린다고 해 봅시다. Memcached는 문자열 안을 들여다볼 줄 모르니, 랭킹 1,000개 전체를 꺼내 오고(GET), 앱에서 풀어서 점수를 고치고 다시 정렬하고, 다시 문자열로 만들어 통째로 저장(SET)해야 합니다. **숫자 하나 바꾸려고 1,000개를 왕복시키는 셈입니다.** 게다가 꺼낸 뒤 저장하기 전 사이에 다른 서버가 끼어들면, 나중에 저장한 쪽이 덮어써서 판매 하나가 사라집니다. 그리고 재시작하면 데이터가 사라지고, 복제도 없습니다. 장면 ③의 문제를 한 층 아래로 옮겼을 뿐인 거죠.

비유하면 Memcached는 모두가 같이 쓰는 사물함은 만들었는데, 칸막이가 없어서 물건 하나 바꾸려면 통째로 꺼냈다 넣어야 하고, 정전되면 다 사라지는 사물함입니다.

17쪽 · 공유하면서 자료구조와 운영 기능까지

2009년에 나온 레디스는 이 빈칸을 채웠습니다. 표를 위에서부터 보면, 모든 서버가 같은 값을 보는 건 Memcached와 같고, 그 아래부터 다릅니다. List, Hash, Sorted Set 같은 자료구조로 일부만 고칠 수 있고, INCR로 숫자를 정확히 올리고, SET NX EX로 락을 걸고, RDB와 AOF로 재시작해도 데이터를 유지하고, Replication으로 한 대가 죽어도 버티고, Cluster로 나눠 담고, Pub/Sub으로 서버끼리 알릴 수 있습니다.

18쪽 · 읽고·고치고·쓰기를 레디스 안에서 한 번에

이 슬라이드가 레디스의 핵심입니다. 부제 그대로, **숫자를 꺼내 와서 앱에서 고치지 말고, 레디스에게 "1 올려 줘"라고 시킨다**는 뜻입니다.

왼쪽을 보시면 조회수가 100일 때 A와 B가 동시에 GET하면 둘 다 100을 받고, 각자 101을 SET합니다. 두 번 올렸는데 101입니다. 공용 저장소에 숫자를 뒀는데도 계산을 앱에서 하니까 틈이 생긴 겁니다.

오른쪽처럼 `INCR`을 쓰면 읽고 더하고 쓰는 게 레디스 안에서 한 명령으로 끝납니다. 레디스는 명령을 한 줄로 세워서 하나씩 처리하니까, A의 INCR이 끝나기 전엔 B의 INCR이 시작되지 않습니다. 결과는 정확히 102입니다. 이렇게 중간에 쪼개지지 않는 걸 원자적이라고 합니다.

칠판 비유로 설명하면 좋습니다. 두 사람이 칠판 숫자를 수첩에 베껴 가서 1을 더해 다시 쓰면 둘 다 101을 씁니다. 칠판 앞 관리인에게 "1 올려 주세요"라고 부탁하면 관리인이 한 사람씩 처리하니 102가 됩니다. 레디스가 그 관리인입니다. 장면 ⑤의 재고도 DECR 결과가 0 미만이면 품절 처리하면 정확히 한 명만 삽니다.

아래 네 박스도 같은 원리입니다. 포인트만 올리기, 최근 본 상품 10개 유지, 좋아요 확인, 랭킹 점수 올리기를 전부 레디스 안에서 한 번에 합니다.

19~20쪽 · 우리 과제

19쪽 · 해시 테이블은 모든 자료구조의 입구다

레디스는 두 층으로 생각하면 쉽습니다. 바깥 층은 "키로 값을 찾는 곳"이고, 안쪽 층은 찾은 값의 모양, 즉 String, List, Set 같은 자료구조입니다. 어떤 명령이든 먼저 키를 찾고(①), 그다음 값을 다룹니다(②). `GET price:키보드`도, `ZINCRBY best 1 키보드`도, `LPUSH recent:kim 키보드`도 전부 ①을 거칩니다. **우리가 만드는 건 이 바깥 층, 해시 테이블 하나입니다.**

왜 하필 해시 테이블이냐면, 리스트로 만들면 처음부터 비교해야 해서 키가 늘수록 느려지고(O(N)), 트리는 O(log N)이지만 우리가 쓰지 않는 정렬에 비용을 냅니다. 해시 테이블은 키로 칸 번호를 바로 계산하니 O(1)입니다. "키로 바로 찾기"에 딱 맞습니다.

20쪽 · 바깥 층과 캐시에 꼭 필요한 기능만 만든다

표는 레디스의 기능과 우리 단계를 짝지은 겁니다. 키로 값 찾기는 2, 3단계 해시 테이블과 칸 수 확장, 만료는 4단계 지연 삭제와 13단계 주기적 정리, 동시 접근은 5단계 lock, INCR은 12단계, 데이터 유지는 14단계(선택), DB 앞 캐시로 쓰기는 8단계 데모 API와 9단계 성능 비교입니다.

동시 접근 줄은 짚어 주세요. 레디스는 싱글 스레드라 명령이 하나씩 처리되지만, 우리 구현은 여러 스레드가 동시에 들어올 수 있어서 lock으로 같은 효과를 냅니다. 18쪽에서 본 "아무도 못 끼어든다"를 우리는 lock으로 만드는 겁니다. Sorted Set, List, Replication, Cluster는 해시 테이블 위에 얹는 기능이라 이번 범위에서는 하지 않습니다.

21~24쪽 · 실무와 정리

21쪽 · Cache-Aside: 레디스를 먼저, 없으면 DB

실무에서 가장 흔한 쓰는 법입니다. 읽을 때는 레디스에 먼저 물어보고, 있으면 바로 돌려줍니다. 없으면 DB에서 읽어서 레디스에 TTL과 함께 넣고 돌려줍니다. 3쪽의 로컬 캐시 코드와 모양이 똑같고, `cache`가 레디스로 바뀐 것뿐입니다.

쓸 때가 중요합니다. DB를 고친 뒤 레디스의 그 키를 지웁니다. 그러면 다음 읽기 때 DB에서 새 값으로 다시 채워집니다. **이 "쓰기 때 키를 지운다"가 장면 ①을 실제로 막는 방법입니다.** 원본은 항상 DB이고, 레디스는 빠른 사본입니다.

22쪽 · 언제 로컬 캐시, 언제 레디스?

레디스가 로컬 캐시를 완전히 대체하는 건 아닙니다. 아래 판단 질문 하나로 정리됩니다. **"이 데이터, 서버마다 따로 들고 있어도 괜찮은가?"**

카테고리 목록처럼 거의 안 바뀌면 서버 세 대가 각자 들고 있어도 셋 다 똑같으니 괜찮습니다. 이건 로컬 캐시가 더 빠릅니다. 가격처럼 자주 바뀌면, 바뀌는 순간 서버마다 달라지니 안 괜찮습니다. 쿠폰 발급 수처럼 모두가 하나의 값을 같이 세야 하면 처음부터 안 괜찮습니다. 표에서 "부적합"과 "불가능"이 나뉘는 게 이 차이입니다.

그래서 실무에서는 셋을 겹쳐 씁니다. 가장 빠른 로컬 캐시, 공유되는 레디스, 원본인 DB 순서입니다.

23쪽 · 레디스도 공짜는 아니다

네 가지 비용을 짚고 넘어가겠습니다. 첫째, 네트워크를 탑니다. 로컬은 나노초, 레디스는 약 1ms입니다. DB보다는 훨씬 빠르지만 로컬보다는 느립니다. 그래서 HTTP 대신 RESP라는 가벼운 자체 프로토콜을 씁니다. 둘째, 모든 서버가 레디스 한 대에 의존하니 새로운 단일 장애점이 됩니다. Replication과 Sentinel이 필요한 이유입니다. 셋째, DB를 대체하지 않습니다. 영속성 설정을 해도 마지막 몇 초에서 몇 분은 잃을 수 있으니 원본은 DB에 둡니다. 넷째, RAM에 저장하니 꽉 차면 오래 안 쓴 키부터 지우는 식의 정책에 따라 키가 사라집니다.

24쪽 · 정리: 상태를 서버 밖으로

오늘 이야기를 한 줄로 줄이면 "상태를 서버 밖으로"입니다. 그러면 앱 서버가 몇 대든 모두가 같은 상태를 보고, DB는 보호받습니다. 레디스는 그 공용 저장소에 자료구조와 원자적 연산까지 얹었습니다.

흐름으로 다시 보면, 로컬 캐시는 빠르지만 공유가 안 됐고, Memcached는 공유는 됐지만 문자열만 됐고, 레디스는 공유에 자료구조와 운영 기능까지 갖췄습니다. 감사합니다.

예상 질문과 답

질문	짧은 답
로드밸런서는 꼭 순서대로 나눠 주나요?	기본값은 라운드 로빈이지만, 가장 한가한 서버로 보내는 최소 연결이나 가중 랜덤도 고를 수 있습니다. 어떤 방식이든 사용자를 기억하지 않는다는 점은 같고, 기억하게 하는 게 스티키 세션입니다.
스티키 세션 쓰면 되지 않나요?	로그아웃(②)만 막습니다. 그 서버가 죽으면 세션이 사라지고, 부하가 쏠리고, 관리자와 고객이 다른 서버에 묶이면 가격 문제(①)는 그대로입니다.
가격 바꾼 서버가 다른 서버들에게 캐시 지우라고 알리면요?	가능은 하지만 모든 서버가 서로를 알아야 하고, 알림이 누락되면 다시 어긋납니다. 서버가 늘수록 복잡해지니 한 곳에 두는 게 단순합니다. 레디스 Pub/Sub이 그 알림 용도로도 쓰입니다.
레디스가 죽으면 어떻게 되나요?	모든 서버가 의존하니 단일 장애점이 됩니다. 그래서 실무에서는 Replication으로 복제본을 두고 Sentinel이나 Cluster로 자동 전환합니다.
Memcached는 INCR이 없나요?	incr/decr과, 읽은 뒤 아무도 안 바꿨을 때만 저장하는 cas가 있습니다. 다만 숫자 외의 구조는 통째로 주고받아야 하고 재시도 로직을 앱이 짜야 해서 표에 △로 표시했습니다.

질문	짧은 답
재고는 DB 트랜잭션으로도 막을 수 있지 않나요?	맞습니다. 다만 선착순처럼 순간에 요청이 몰리면 DB 잠금 경쟁이 심해져서, 앞단에서 레디스 DECR 로 먼저 거르고 DB에는 확정된 주문만 보내는 식으로 많이 씁니다.
레디스가 싱글 스레드면 느리지 않나요?	데이터가 메모리에 있고 명령 하나가 매우 짧아서 한 줄로 처리해도 빠릅니다. 오히려 한 줄로 처리하기 때문에 INCR 같은 원자적 연산이 공짜로 됩니다.